

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Universidad Nacional de La Plata



Facultad de Informática

Ejemplo de aplicación

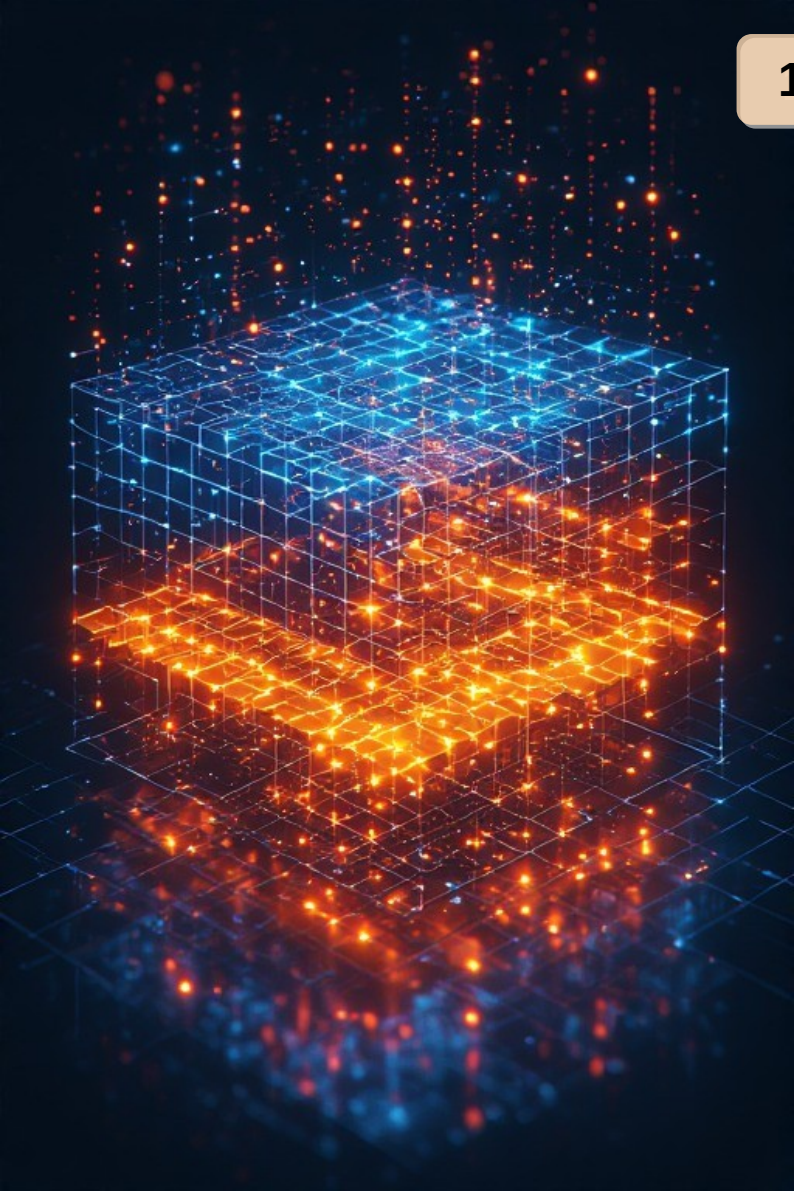
Multiplicación de matrices en CUDA (Optimizado)



1

Multiplicación de matrices

Mejoras por el uso de memoria compartida



1

Multiplicación de matrices

Mejoras por el uso de memoria compartida



Multiplicación de matrices

Solución GPU - Problemas

4

- Un bloque de hilos necesitará acceder a ciertas posiciones de las matrices A y B .

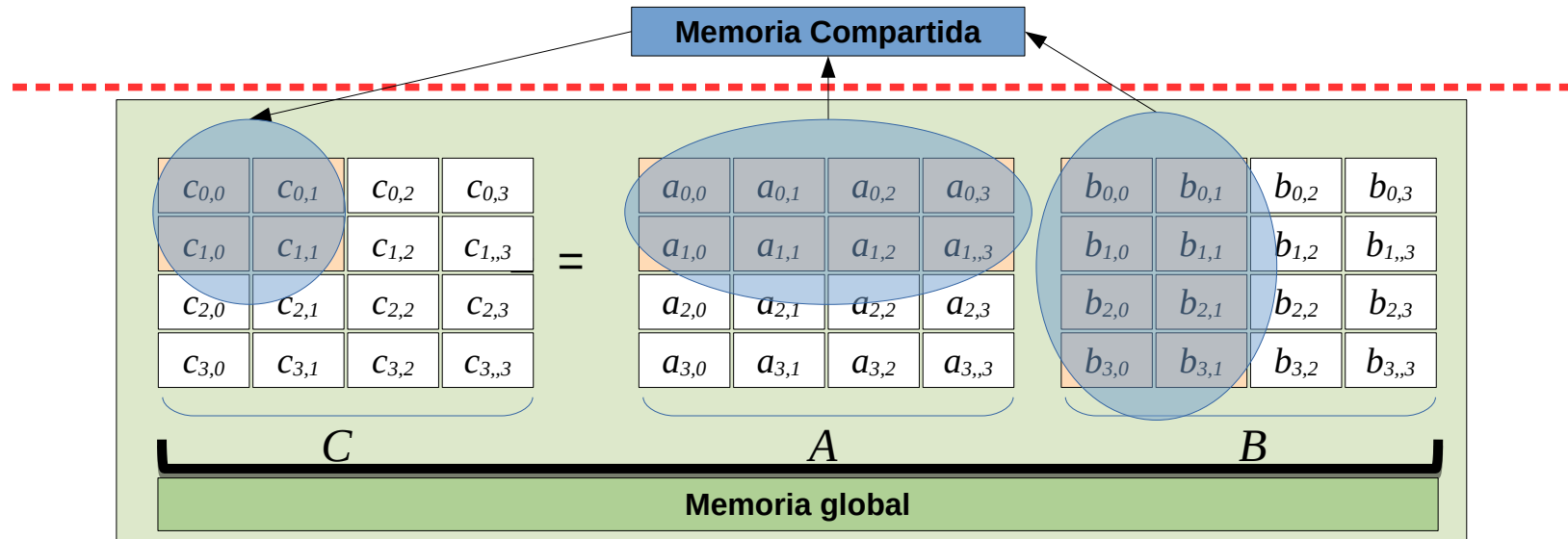
$$\begin{matrix} \begin{matrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{matrix} & = & \begin{matrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{matrix} & \begin{matrix} b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \end{matrix} \\ C & & A & B \end{matrix}$$

- Varios hilos de un mismo bloque acceden varias veces a las mismas posiciones en memoria global.
- Estos accesos a memoria global resultan ser costosos.
- Podemos evitar la repetición de accesos al mismo dato en memoria global utilizando una memoria más rápida como la memoria compartida.

Multiplicación de matrices

Mejoras por el uso de memoria compartida

- La estrategia consiste en:
 - Traer, de forma coalescente, desde memoria global a memoria compartida, los datos necesarios correspondientes a las filas de A y columnas de B .
 - Procesarlos en memoria compartida.
 - El resultado en memoria compartida almacenarlo en la memoria global.

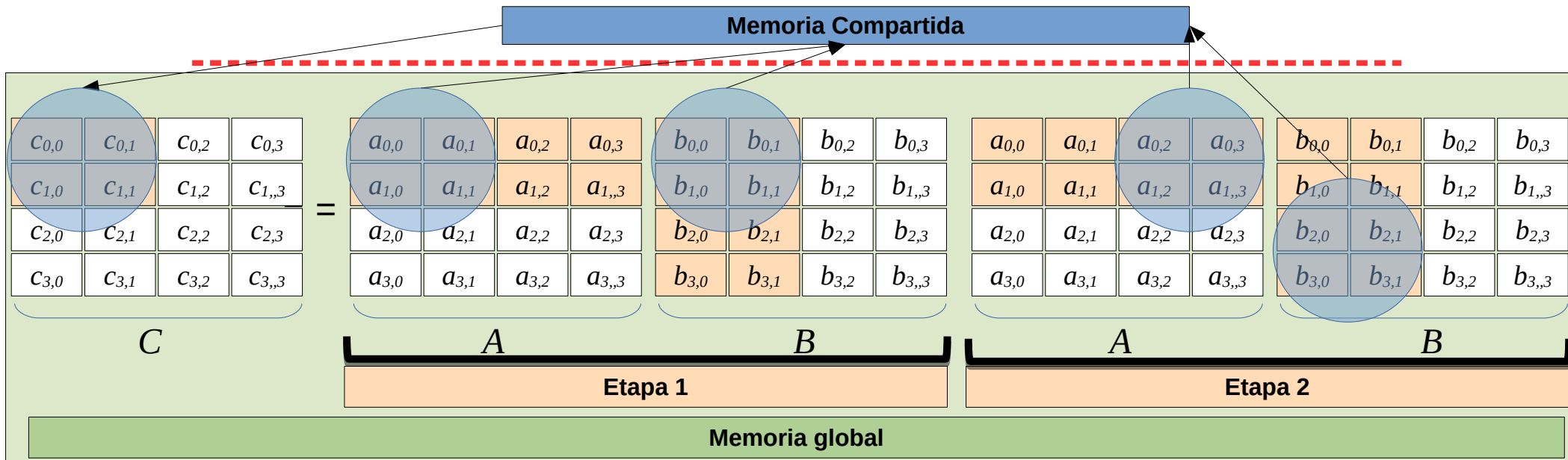


Multiplicación de matrices

Mejoras por el uso de memoria compartida

6

- Por las limitaciones de la memoria compartida y para maximizar el rendimiento, las filas de A y las columnas de B se traen y calculan por etapas, de a **TILES**.
- Al completar el cálculo de todas las etapas, se almacena el resultado que reside en memoria compartida en la memoria global.



Multiplicación de matrices

Mejoras por el uso de memoria compartida - Implementación

- Suponemos que cada bloque bidimensional CUDA contiene $\text{BLOCK_SIZE} \times \text{BLOCK_SIZE}$ hilos.

```
__global__ void mm_optimizado(float *C, float *A, float *B, int N){
    __shared__ As[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ Bs[BLOCK_SIZE][BLOCK_SIZE];

    int fila = blockIdx.y * BLOCK_SIZE + threadIdx.y;
    int columna = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    float c_temp=0.0;
    for(int etapa=0; etapa< N/BLOCK_SIZE; etapa++){
        As[threadIdx.y][threadIdx.x] = A[fila*N + etapa*BLOCK_SIZE + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(etapa*BLOCK_SIZE + threadIdx.y)*N + columna];

        __syncthreads();

        for( k=0 ; k<BLOCK_SIZE ; k++ )
            c_temp += As[fila,k] * B[k,columna];

        __syncthreads();

    }
    C[fila*N+columna]= c_temp;
}
```

Espacio en memoria compartida para guardar cada TILE

Calcula la fila y columna sobre la que trabajará cada hilo

*Lectura coalescente de memoria global a memoria compartida
Cada hilo trae un dato por matriz.*

Cálculo. En cada etapa deja resultados parciales en c_temp

*Escritura coalescente
Cada hilo escribe su resultado almacenado en c_temp a la memoria global*

